

Hand movement controlled RC Car

INTRODUCTION

The project helps to control RC car with nothing but simple hand movements and tilts. Instead of learning to control using joystick or button remotes, this makes it easy to intuitively control the movement of a RC car.

COMPONENTS USED

The components used for different parts of the project along with their use case are listed below:

Hand Movement Remote Control Components:

- ESP32 : Works as the brain of the controller to calculate angle tilt and sends the commands according to the direction hand is tilting towards.
- MPU6050 : Sends input data to the ESP for calculation. It has the ability to measure acceleration due to gravity and gyro for all three dimensions, we use that information to calculate hand tilt.

Car Components:

- ESP32 : Works as the brain of the car and receives commands from the Remote Controller to apply necessary adjustments to motors.
- BO motors : Works as the back tires to drive the car.
- Servo motor : Attached to axle of the front tires to change direction.
- Dual H-bridge Motors Driver : Uses PWM signals to control the two BO motors . It controls both speed and directions from the command of the ESP.

PROBLEM STATEMENT

Traditional remote controlled vehicles and robotic systems are generally operated using joystick or button based controllers. While effective, such control methods often require users to learn specific button layouts and develop coordination skills before they can operate the system efficiently. This can create a barrier for new users and reduce the intuitiveness of the control experience. Furthermore, conventional controllers may feel less natural when precise directional movements are required, especially for beginners.

As robotics and automation systems become increasingly common in education, research, and consumer applications, there is a growing need for control systems that are simple, intuitive, and easy to adopt without extensive training. Therefore,

this project aims to explore an alternative method of human-machine interaction using natural hand movements and gestures as control inputs.

PROPOSED SOLUTION

To address the limitations of traditional controllers, this project implements a hand movement controlled RC car using an ESP32 microcontroller and an MPU6050 motion sensor. The system detects the tilt and orientation of the user's hand and converts these movements into movement commands for the vehicle.

The controller continuously measures roll and pitch angles using accelerometer and gyroscope data from the MPU6050. These values are processed by the ESP32 and transmitted wirelessly to the RC car using Wi-Fi communication. The car then interprets the received commands and performs the corresponding actions such as moving forward, backward, turning left, turning right, or stopping.

By allowing users to control the vehicle through natural hand motions, the system provides a more intuitive and engaging control experience while demonstrating the practical application of motion sensing and wireless communication technologies.

ADVANTAGES OF THE PROPOSED SYSTEM

1. Intuitive Operation

The vehicle can be controlled through simple hand tilts, making it easier for new users to understand and operate compared to conventional joystick-based controllers.

2. Reduced Learning Curve

Users do not need to memorize button layouts or control combinations. The direction of hand movement directly corresponds to the direction of vehicle movement.

3. Improved User Engagement

The gesture-based control method creates a more interactive and immersive experience, making the system suitable for demonstrations, educational projects, and robotics exhibitions.

4. Natural Human-Machine Interaction

The project demonstrates a more natural form of communication between

humans and machines by utilizing body movements instead of physical buttons.

5. Expandability

The same control concept can be applied to various robotic platforms such as robotic arms, mobile robots, wheelchairs, drones, and automated vehicles with further development and optimization.

CODES USED FOR THE PROJECT

The codes used for this project is written below. While there maybe slight changes to different values for fine-tuning and other requirements,the major working of the project shall remain unchanged.

CodesforRemoteController

```
#include <Wire.h>

#include <WiFi.h>


#define MPU_ADDR 0x68


const char* ssid = "RC_CAR";
const char* password = "12345678";
const char* host = "192.168.4.1";


WiFiClient client;


// MPU variables
int16_t accX, accY, accZ;
int16_t gyroX, gyroY, gyroZ;


float roll = 0, pitch = 0;
float rollAcc, pitchAcc;
float gyroRollRate, gyroPitchRate;
```

```
unsigned long lastTime = 0;

float dt;

float alpha = 0.98;

String lastCommand = "";

void connectToServer() {
  while (!client.connect(host, 80)) {
    Serial.println("Retrying connection...");
    delay(500);
  }
  Serial.println("Connected to ESP server");
}

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi Connected");
  Serial.println(WiFi.localIP());

  connectToServer();
}
```

```

// MPU wake
Wire.beginTransmission(MPU_ADDR);
Wire.write(0x6B);
Wire.write(0);
Wire.endTransmission(true);

lastTime = micros();
}

void loop() {
  // TIME STEP
  unsigned long now = micros();
  dt = (now - lastTime) * 1e-6;
  lastTime = now;

  // READ MPU
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x3B);
  Wire.endTransmission(false);
  Wire.requestFrom(MPU_ADDR, 14, true);

  accX = Wire.read() << 8 | Wire.read();
  accY = Wire.read() << 8 | Wire.read();
  accZ = Wire.read() << 8 | Wire.read();
  Wire.read(); Wire.read();

  gyroX = Wire.read() << 8 | Wire.read();
  gyroY = Wire.read() << 8 | Wire.read();
  gyroZ = Wire.read() << 8 | Wire.read();

```

```

float Ax = accX / 16384.0;
float Ay = accY / 16384.0;
float Az = accZ / 16384.0;

gyroRollRate = gyroX / 131.0;
gyroPitchRate = gyroY / 131.0;

rollAcc = atan2(Ay, sqrt(Ax*Ax + Az*Az)) * 180 / PI;
pitchAcc = atan2(-Ax, sqrt(Ay*Ay + Az*Az)) * 180 / PI;

roll += gyroRollRate * dt;
pitch += gyroPitchRate * dt;

roll = alpha * roll + (1 - alpha) * rollAcc;
pitch = alpha * pitch + (1 - alpha) * pitchAcc;

// ----- COMMAND LOGIC -----
String cmd = "S";

if (roll > 40) cmd = "R";
else if (roll < -20) cmd = "L";
else if (pitch > 0) cmd = "F";
else if (pitch < -40) cmd = "B";

// ----- SEND ONLY IF CHANGED -----
if (cmd != lastCommand && client.connected()) {
    client.println(cmd); // IMPORTANT: newline
    Serial.println(cmd);
}

```

```

        lastCommand = cmd;
    }

    // ----- RECONNECT IF DISCONNECTED -----
    if (!client.connected()) {
        Serial.println("Reconnecting...");
        client.stop();
        connectToServer();
    }

    delay(10);
}

```

Codes for Car

```

#include <WiFi.h>

const char* ssid = "RC_CAR";
const char* password = "12345678";

WiFiServer server(80);
WiFiClient client;

char lastCmd = 'd';
String buffer = "";

// Motor pins
#define IN1 18
#define IN2 19
#define IN3 21
#define IN4 22
#define ENA 5
#define ENB 23

char command;

void setup() {
    Serial.begin(115100);

    WiFi.softAP(ssid, password);

```

```

Serial.println("AP Started");
Serial.println(WiFi.softAPIP());

server.begin();

pinMode(IN1, OUTPUT);
pinMode(IN2, OUTPUT);
pinMode(IN3, OUTPUT);
pinMode(IN4, OUTPUT);

// NEW ESP32 PWM API (Core v3+)
ledcAttach(ENA, 1000, 8); // pin, freq, resolution
ledcAttach(ENB, 1000, 8);

startCar();
}

void loop() {

  // Accept new client
  if (!client || !client.connected()) {
    client = server.available();

    if (client) {
      Serial.println("Client Connected");
    }
  }

  // Read incoming data
  if (client && client.connected()) {

    while (client.available()) {
      char c = client.read();

      // command is single char OR newline
      if (c == '\n' || c == '\r') {
        if (buffer.length() > 0) {
          lastCmd = buffer[0]; // take first character only
          buffer = "";
        }
      } else {
        buffer += c;
      }
    }
  }
  else {
    // only STOP when disconnected
    lastCmd = 'S';
  }
}

```

```

    }

    applyMotor(lastCmd);
}

void applyMotor(char cmd) {
    Serial.println(cmd);
    switch (cmd) {
        case 'F': forward(); break;
        case 'B': backward(); break;
        case 'L': left(); break;
        case 'R': right(); break;
        case 'S': stopCar(); break;
    }
}

// ----- MOTION FUNCTIONS -----
void startCar() {
    ledcWrite(ENA, 100);
    ledcWrite(ENB, 100);

    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
}

void forward() {
    ledcWrite(ENA, 250);
    ledcWrite(ENB, 250);

    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);

    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
}

void backward() {
    ledcWrite(ENA, 250);
    ledcWrite(ENB, 250);

    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);

    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
}

```

```

void left() {
  ledcWrite(ENA, 200);
  ledcWrite(ENB, 250);

  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);

  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
}

void right() {
  ledcWrite(ENA, 250);
  ledcWrite(ENB, 200);

  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);

  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
}

void stopCar() {
  ledcWrite(ENA, 0);
  ledcWrite(ENB, 0);

  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
}

```

LIMITATIONS OF THE PROJECT

1. Limited Communication Range

Since the system relies on Wi-Fi communication, the operating range is restricted by the coverage and signal strength of the wireless connection.

2. Communication Latency

Small delays may occur between hand movement and vehicle response due to wireless data transmission and processing time.

3. Control Jitter and Sensitivity

The system is currently not fully fine-tuned. Minor hand tremors or sudden movements may result in unwanted control inputs, affecting smooth operation.

4. Power Consumption

Continuous operation of Wi-Fi modules, sensors, motors, and servo motors increases power consumption and limits battery life.

5. Environmental Interference

Wireless performance may be affected by interference from other Wi-Fi networks or electronic devices operating in the same frequency range.

6. Limited Precision

Compared to high-quality joystick controllers, gesture-based controls may provide less precise movement control, especially during complex maneuvers.

7. Mechanical Constraints

The performance of the system depends on the accuracy of sensor calibration, steering mechanism alignment, and overall vehicle construction quality.

ADVERSITIES FACED DURING THE PROJECT AND THEIR SOLUTIONS

During the development of the Hand Movement Controlled RC Car, several technical challenges were encountered. These challenges required repeated testing, troubleshooting, and modifications to both hardware and software components before a stable demonstration could be achieved.

1. Steering System Integration Challenges

One of the major challenges encountered during the project was integrating the servo motor-based steering mechanism with the vehicle while maintaining stable system operation. During testing, the addition of the servo introduced power-related instability that affected the overall reliability of the system.

As the primary objective of the project was to successfully demonstrate hand gesture-based wireless control, the team decided to temporarily remove the servo steering system from the final demonstration version and focus on ensuring reliable communication and vehicle movement. This allowed the project goals to be achieved while identifying an important area for future improvement.

2. Chassis Design and Component Integration Challenges

Integrating all hardware components within the limited space of the chassis proved to be one of the most significant challenges during development. The BO motors, battery pack, ESP32 modules, motor driver, wiring, and switching components all needed to be accommodated while maintaining a compact and design.

The BO motors occupied a considerable portion of the available space, making it difficult to find suitable mounting locations for the batteries and electronic components. Proper wire routing also became challenging due to the limited clearance inside the chassis. Several arrangements were tested before a layout was found that balanced weight distribution, accessibility, and functionality.

Additional difficulties were encountered while organizing the battery pack and ESP32 modules. The batteries required secure mounting to prevent movement during operation, while the ESP32 modules needed to remain accessible for programming, troubleshooting, and wiring modifications. Multiple design revisions were made to optimize component placement and reduce interference between different parts of the system.

To improve maintainability, the chassis was designed with a removable upper section utilizing slot-and-tab construction. This approach allowed the top cover to be easily detached whenever access to internal components was required. As a result, future repairs, wiring adjustments, battery replacement, and system upgrades can be performed without disassembling the entire vehicle structure.

3. Sensor Calibration and Control Accuracy

The MPU6050 sensor readings were sensitive to small hand movements and vibrations. This occasionally resulted in unintended control commands and inconsistent vehicle behavior.

To improve control stability, accelerometer and gyroscope data were combined using

a complementary filter. Threshold values were also introduced to distinguish intentional hand movements from minor fluctuations, resulting in more reliable control inputs.

4. Wireless Communication Reliability

Maintaining stable communication between the controller and the RC car was another challenge during development. Temporary disconnections and inconsistent responses were observed during testing.

The communication system was optimized through software modifications and repeated testing of the Wi-Fi connection. The ESP32 Wi-Fi configuration was adjusted to improve connection stability and responsiveness.

5. System Fine-Tuning

Different users naturally tilt their hands at slightly different angles. As a result, finding suitable threshold values for forward, backward, left, and right commands required extensive experimentation.

Multiple test runs were conducted and control thresholds were gradually refined until a balance between responsiveness and usability was achieved.

Through systematic testing, troubleshooting, and continuous improvements, these challenges were successfully addressed, resulting in a functional hand movement controlled RC car capable of demonstrating intuitive wireless vehicle control.

COMPLETED PROJECT PICTURES

